

An overview of RAMCloud

RAMCloud is a storage system that stores data in the DRAM of thousands of servers within a data centre, as shown in Figure 1.

Since the information is kept in the DRAM at all times, the access rate is very low, and provides 100-1000x lower latency than disk-based systems and 100-1000x greater throughput. Most Web applications grow over a period of time and will require more servers to store their data. RAMCloud will scale automatically to support the growing number of servers added to the system.

RAMCloud uses DRAM, which has volatile memory, i.e., the data is lost when the power is removed. However, applications require storage systems to provide a high level of data durability and availability. RAMCloud uses a technique called *buffered logging* to maintain durability. In this approach, a single copy of each data object is stored in the DRAM of a primary server on the disks of two or more backup servers; each acts as both primary and backup server. When a write operation is performed, the primary server updates its DRAM and forwards log entries to the backup servers, where they are stored temporarily in the DRAM of the backup server. The backup server collects log entries into batches that can be written efficiently to a log on disk. Once log entries have been written to disk they can be removed from the backup's DRAM.

Buffered logging allows both reads and writes to proceed at DRAM speeds while still providing durability. Power failures can be handled by committing each write operation to a stable storage.

The data model

The data model for a storage system governs how data is collected, stored, integrated and put to use. There are three main factors that we need to decide on, prior to selecting the type of data model:

- The nature of the basic objects stored in the system.
- How basic objects are organised into higher-level structures; for example, we can either just have key-value pairs or some sort of aggregation.
- Select the methods for naming and indexing of objects when retrieving or modifying objects.

The two common types of data models are the highly structured relational data model and the unstructured data model. RAMCloud prefers an intermediate approach where servers do not impose structure on data but do support aggregation and indexing. It supports any number of tables, each stores multiple objects and these objects are stored as simple key-value pairs. It also provides a simple set of operations for creating and deleting tables, and for reading, writing and deleting objects within a table.

Research challenges

Numerous challenges need to be fixed for RAMCloud

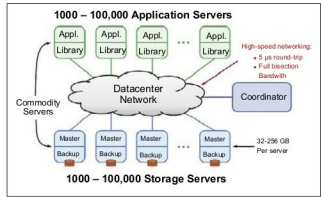


Figure 1: RAMCloud architecture

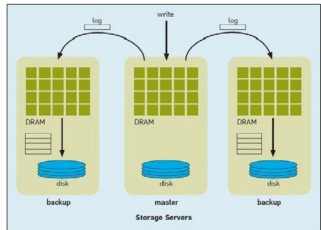


Figure 2: Buffered logging

to be implemented successfully. Given below is a short description of various challenges that researchers are trying to solve.

Consider the applications that use TCP/IP protocol - they have round-trip times for remote procedure calls and high latency in network switches. Also, the flow oriented feature of TCP is of little use to RAMCloud, since individual requests will be relatively small. To improve overall latency, we can either modify or replace the TCP protocol with a UDP protocol. An increasing number of applications are using virtualisation for greater portability. This increases the overheads, since an incoming packet will now need to pass through the virtual machine monitor and a guest operating system before reaching the application, thereby increasing the overall latency. So we need techniques like passing packets directly from the virtual monitor to the application to reduce this overhead.

RAMCloud is implemented by using a large number of servers, but the applications that use them, must see this as a single storage system, i.e., the application must be oblivious to the distribution of the storage system. The primary issue in the distribution and scaling of the system is data placement. An object may be required to be moved to another server to improve the performance. This data movement needs to